

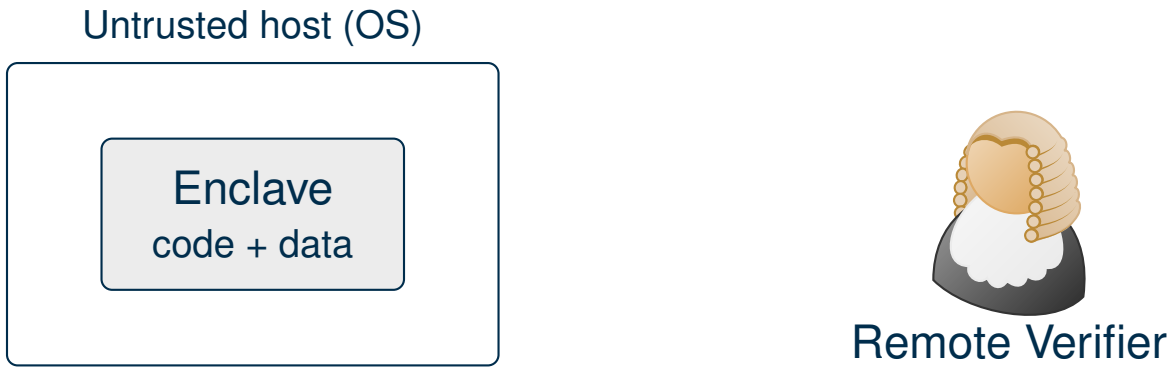
More from Less: Composable General Multi-Party Computation with Global Public Verifiability from a Single Enclave Only

Theory of Cryptography Conference

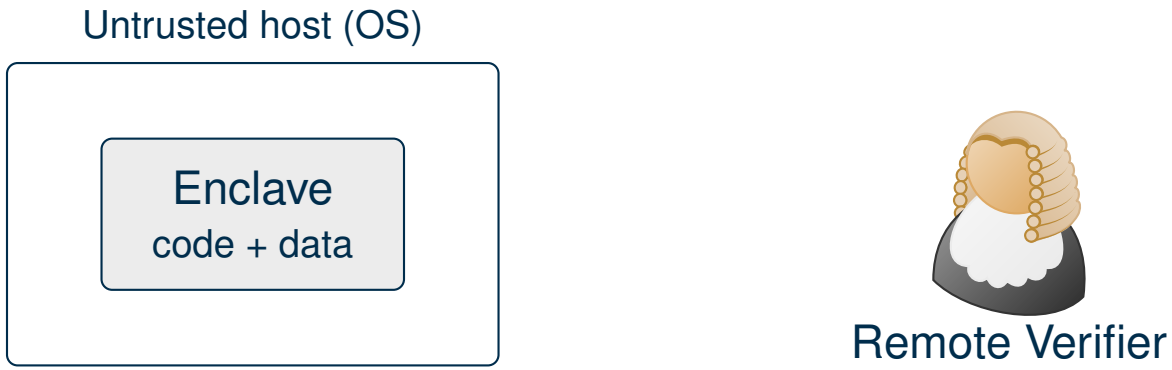
Saskia Bayreuther, Robin Berger, Felix Dörre, Eva Hetzel, Yufan Jiang, Christian Martin, Jeremias Mechler, Jörn Müller-Quade | November 13, 2026

KASTEL

Secure Enclaves

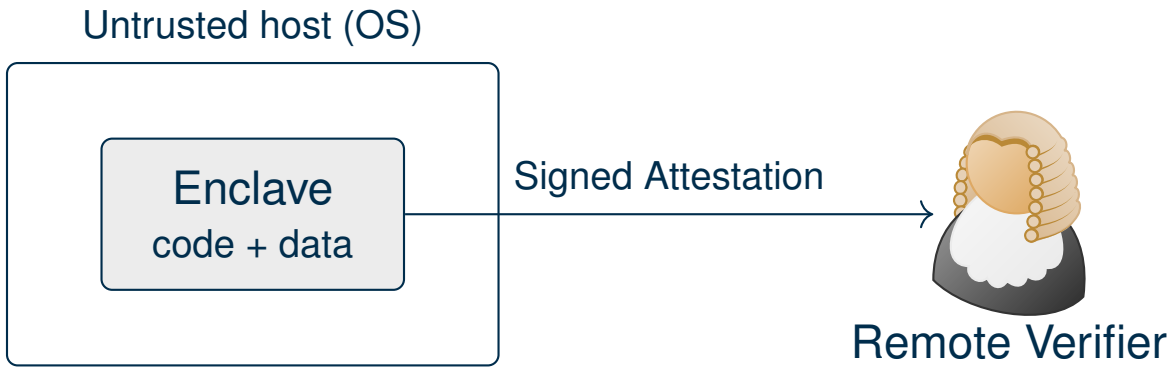


Secure Enclaves



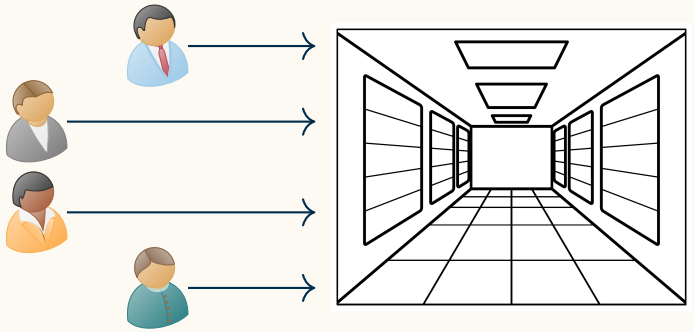
- **Isolated execution:** the host can neither read nor modify the enclave's code and data (confidentiality and integrity)

Secure Enclaves



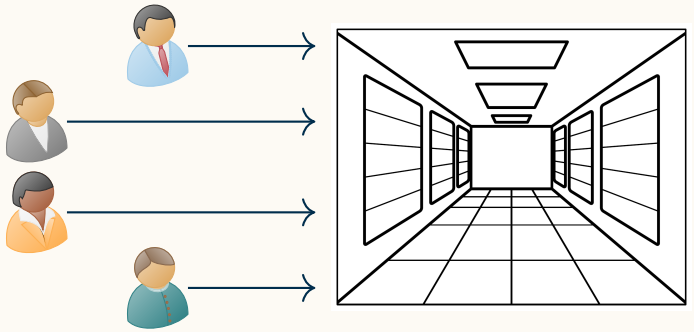
- **Isolated execution:** the host can neither read nor modify the enclave's code and data (confidentiality and integrity)
- **Remote attestation:** a hardware-rooted signature states which code is running and can bind its output

How Enclaves Are Used in Practice

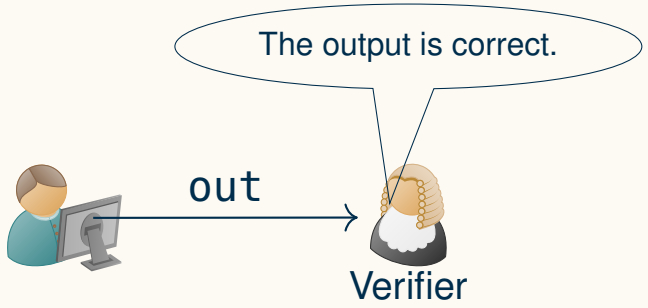


Single-Enclave Confidential Cloud Computing

How Enclaves Are Used in Practice

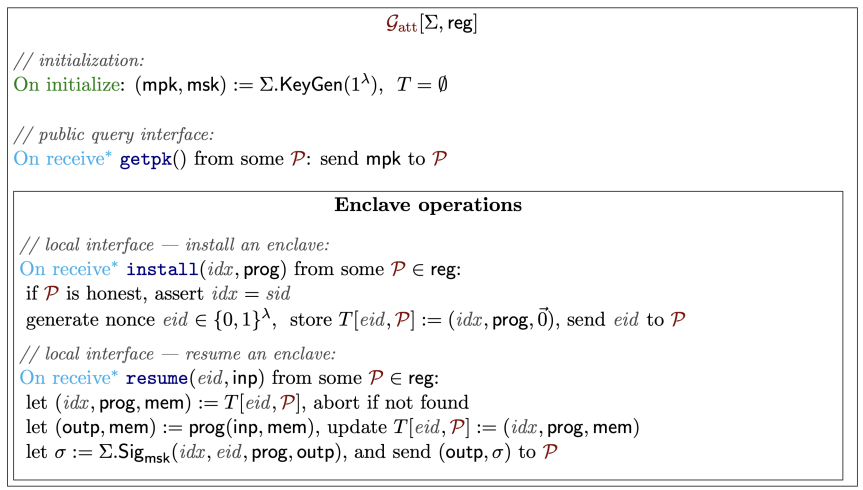


Single-Enclave Confidential Cloud Computing



Publicly Verifiable Outputs

How Enclaves Are Modeled



$\mathcal{G}_{\text{att}}[\Sigma, \mathsf{reg}]$

// initialization:
On initialize: $(\mathsf{mpk}, \mathsf{msk}) := \Sigma.\mathsf{KeyGen}(1^\lambda)$, $T = \emptyset$

// public query interface:
On receive* `getpk`() from some $\mathcal{P}$: send mpk to $\mathcal{P}$

Enclave operations

// local interface — install an enclave:
On receive* `install`(idx, prog) from some $\mathcal{P} \in \mathsf{reg}$:
if $\mathcal{P}$ is honest, assert $idx = sid$
generate nonce $eid \in \{0,1\}^\lambda$, store $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \vec{0})$, send eid to $\mathcal{P}$

// local interface — resume an enclave:
On receive* `resume`(eid, inp) from some $\mathcal{P} \in \mathsf{reg}$:
let $(idx, \mathsf{prog}, \mathsf{mem}) := T[eid, \mathcal{P}]$, abort if not found
let $(\mathsf{outp}, \mathsf{mem}) := \mathsf{prog}(\mathsf{inp}, \mathsf{mem})$, update $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \mathsf{mem})$
let $\sigma := \Sigma.\mathsf{Sig}_{\mathsf{msk}}(idx, eid, \mathsf{prog}, \mathsf{outp})$, and send (outp, σ) to $\mathcal{P}$



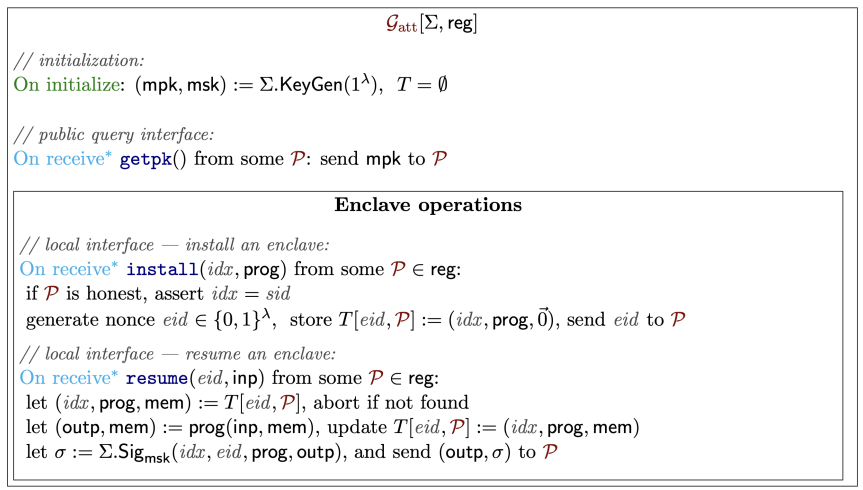
Formal Abstractions for Attested Execution Secure Processors

Rafael Pass, CornellTech — Elaine Shi, Cornell — Florian Tramèr, Stanford

Enclave Functionality $\mathcal{G}_{\text{att}}^{\text{PST}}$[1] in the Universal Composability (UC) Framework

[1] Pass, Shi, and Tramèr 2017.

How Enclaves Are Modeled



$\mathcal{G}_{\text{att}}[\Sigma, \mathsf{reg}]$

// initialization:
On initialize: $(\mathsf{mpk}, \mathsf{msk}) := \Sigma.\mathsf{KeyGen}(1^\lambda)$, $T = \emptyset$

// public query interface:
On receive* `getpk`() from some $\mathcal{P}$: send mpk to $\mathcal{P}$

Enclave operations

// local interface — install an enclave:
On receive* `install`(idx, prog) from some $\mathcal{P} \in \mathsf{reg}$:
if $\mathcal{P}$ is honest, assert $idx = sid$
generate nonce $eid \in \{0,1\}^\lambda$, store $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \vec{0})$, send eid to $\mathcal{P}$

// local interface — resume an enclave:
On receive* `resume`(eid, inp) from some $\mathcal{P} \in \mathsf{reg}$:
let $(idx, \mathsf{prog}, \mathsf{mem}) := T[eid, \mathcal{P}]$, abort if not found
let $(\mathsf{outp}, \mathsf{mem}) := \mathsf{prog}(\mathsf{inp}, \mathsf{mem})$, update $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \mathsf{mem})$
let $\sigma := \Sigma.\mathsf{Sig}_{\mathsf{msk}}(idx, eid, \mathsf{prog}, \mathsf{outp})$, and send (outp, σ) to $\mathcal{P}$

Formal Abstractions for Attested Execution Secure Processors

Rafael Pass, CornellTech
Elaine Shi, Cornell
Florian Tramèr, Stanford

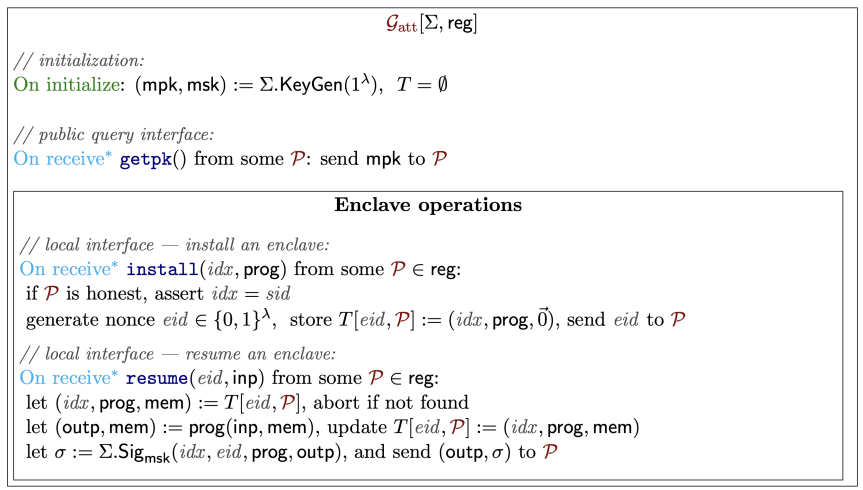
AGATE: Augmented Global Attested Trusted Execution in the Universal Composability framework

Lorenzo Martinico, Markulf Kohlweiss
University of Edinburgh & Input Output

Enclave Functionality $\mathcal{G}_{\text{att}}^{\text{PST}}$[1] in the Universal Composability (UC) Framework

[1] Pass, Shi, and Tramèr 2017.

How Enclaves Are Modeled



$\mathcal{G}_{\text{att}}[\Sigma, \mathsf{reg}]$

// initialization:
On initialize: $(\mathsf{mpk}, \mathsf{msk}) := \Sigma.\mathsf{KeyGen}(1^\lambda)$, $T = \emptyset$

// public query interface:
On receive* `getpk`() from some $\mathcal{P}$: send mpk to $\mathcal{P}$

Enclave operations

// local interface — install an enclave:
On receive* `install`(idx, prog) from some $\mathcal{P} \in \mathsf{reg}$:
if $\mathcal{P}$ is honest, assert $idx = sid$
generate nonce $eid \in \{0,1\}^\lambda$, store $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \vec{0})$, send eid to $\mathcal{P}$

// local interface — resume an enclave:
On receive* `resume`(eid, inp) from some $\mathcal{P} \in \mathsf{reg}$:
let $(idx, \mathsf{prog}, \mathsf{mem}) := T[eid, \mathcal{P}]$, abort if not found
let $(\mathsf{outp}, \mathsf{mem}) := \mathsf{prog}(\mathsf{inp}, \mathsf{mem})$, update $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \mathsf{mem})$
let $\sigma := \Sigma.\mathsf{Sig}_{\mathsf{msk}}(idx, eid, \mathsf{prog}, \mathsf{outp})$, and send (outp, σ) to $\mathcal{P}$



Formal Abstractions for Attested Execution Secure Processors

Practically Efficient Private Set Intersection from Trusted Hardware with Side-Channels

Felix Dörre, Jeremias Mechler(✉), and Jörn Müller-Quade

KASTEL Security Research Labs, Karlsruhe Institute of Technology, Karlsruhe, Germany

ted Trusted Execution ility framework

lf Kohlweiss

Input Output

Enclave Functionality $\mathcal{G}_{\text{att}}^{\text{PST}}$[1] in the Universal Composability (UC) Framework

[1] Pass, Shi, and Tramèr 2017.

Practice and Models Do Not Match





$\mathcal{G}_{\text{att}}[\Sigma, \mathsf{reg}]$

// initialization:
On initialize: $(\mathsf{mpk}, \mathsf{msk}) := \Sigma.\mathsf{KeyGen}(1^\lambda)$, $T = \emptyset$

// public query interface:
On receive* `getpk`() from some $\mathcal{P}$: send mpk to $\mathcal{P}$

Enclave operations

// local interface — install an enclave:
On receive* `install`(idx, prog) from some $\mathcal{P} \in \mathsf{reg}$:
if $\mathcal{P}$ is honest, assert $idx = sid$
generate nonce $eid \in \{0,1\}^\lambda$, store $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \vec{0})$, send eid to $\mathcal{P}$

// local interface — resume an enclave:
On receive* `resume`(eid, inp) from some $\mathcal{P} \in \mathsf{reg}$:
let $(idx, \mathsf{prog}, \mathsf{mem}) := T[eid, \mathcal{P}]$, abort if not found
let $(\mathsf{outp}, \mathsf{mem}) := \mathsf{prog}(\mathsf{inp}, \mathsf{mem})$, update $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \mathsf{mem})$
let $\sigma := \Sigma.\mathsf{Sig}_{\mathsf{msk}}(idx, eid, \mathsf{prog}, \mathsf{outp})$, and send (outp, σ) to $\mathcal{P}$

Practice and Models Do Not Match



- Single-enclave cloud computing
- External verifier validating attestations

$\mathcal{G}_{\text{att}}[\Sigma, \mathsf{reg}]$

// initialization:
On initialize: $(\mathsf{mpk}, \mathsf{msk}) := \Sigma.\mathsf{KeyGen}(1^\lambda)$, $T = \emptyset$

// public query interface:
On receive* `getpk`() from some $\mathcal{P}$: send mpk to $\mathcal{P}$

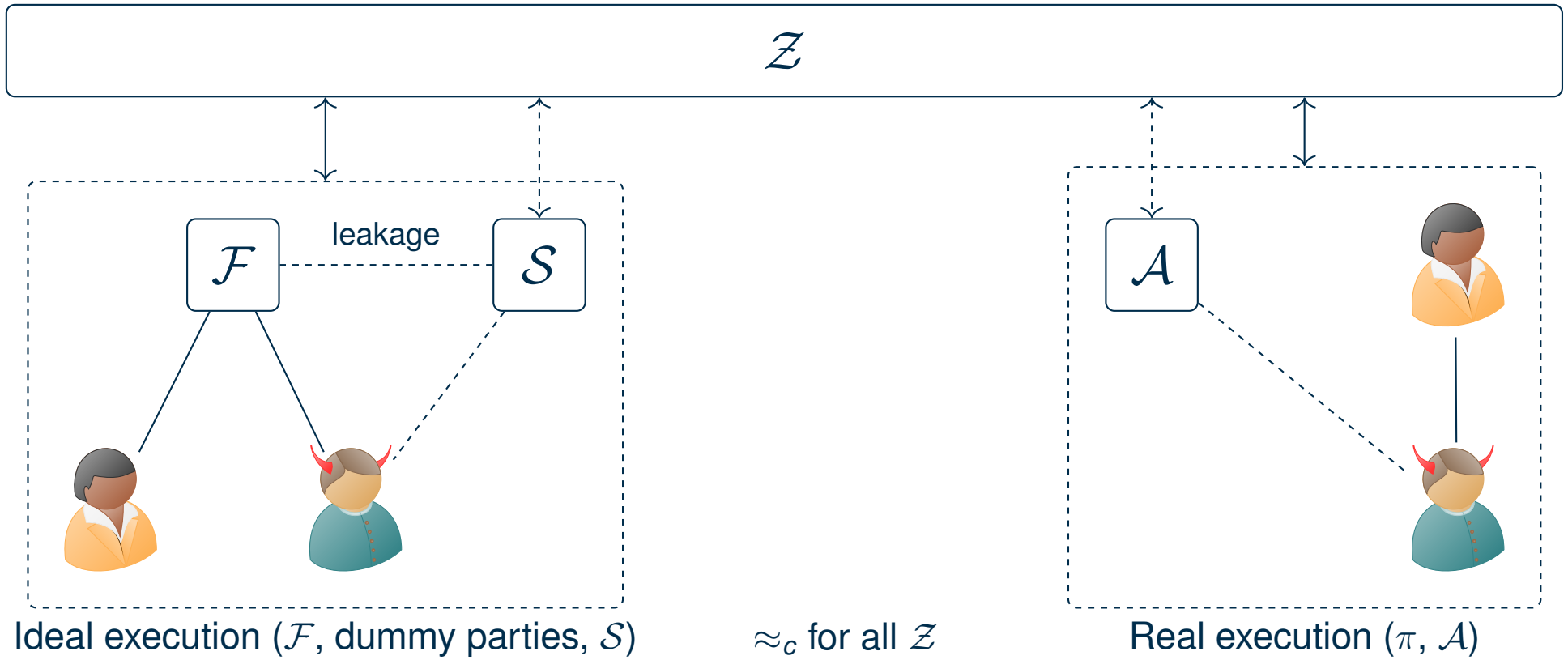
Enclave operations

// local interface — install an enclave:
On receive* `install`(idx, prog) from some $\mathcal{P} \in \mathsf{reg}$:
if $\mathcal{P}$ is honest, assert $idx = sid$
generate nonce $eid \in \{0,1\}^\lambda$, store $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \vec{0})$, send eid to $\mathcal{P}$

// local interface — resume an enclave:
On receive* `resume`(eid, inp) from some $\mathcal{P} \in \mathsf{reg}$:
let $(idx, \mathsf{prog}, \mathsf{mem}) := T[eid, \mathcal{P}]$, abort if not found
let $(\mathsf{outp}, \mathsf{mem}) := \mathsf{prog}(\mathsf{inp}, \mathsf{mem})$, update $T[eid, \mathcal{P}] := (idx, \mathsf{prog}, \mathsf{mem})$
let $\sigma := \Sigma.\mathsf{Sig}_{\mathsf{msk}}(idx, eid, \mathsf{prog}, \mathsf{outp})$, and send (outp, σ) to $\mathcal{P}$

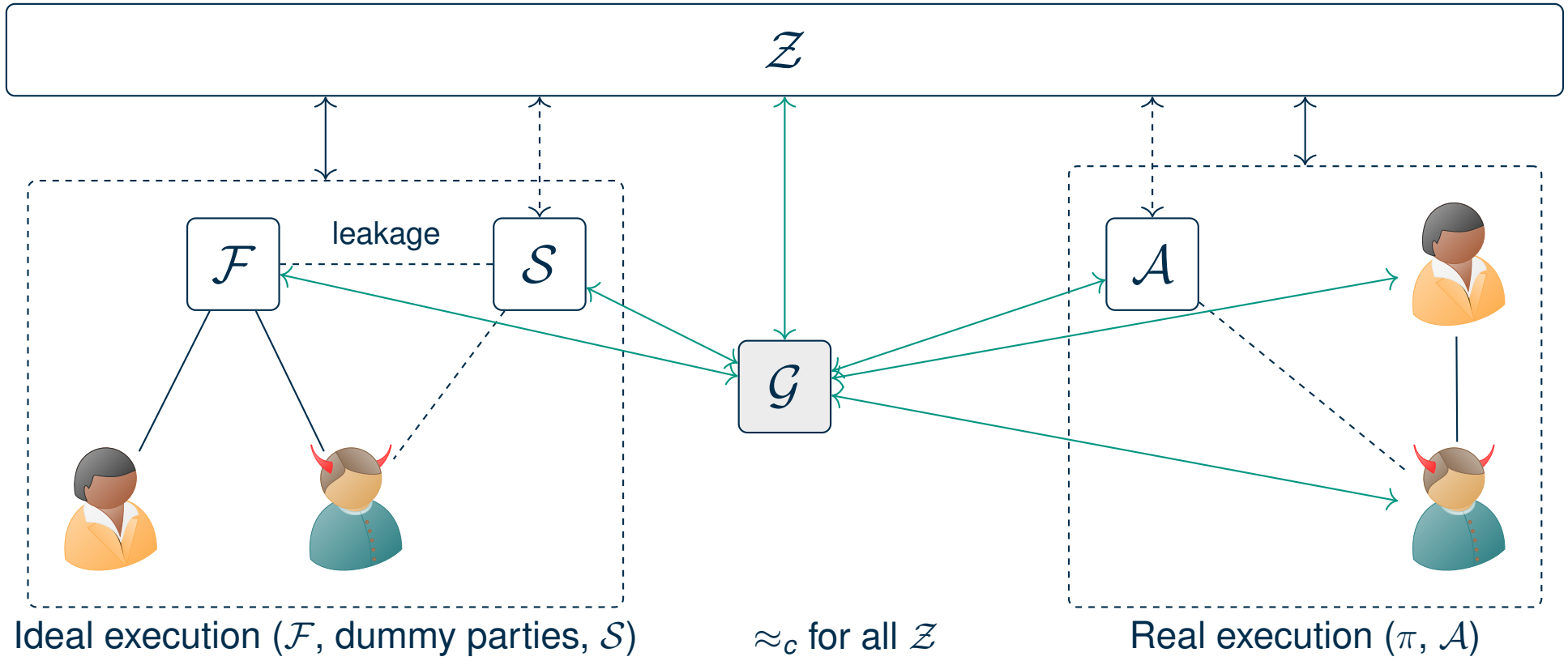
- All parties need enclaves
- Universal Composability contradicts public verifiability

Universal Composability



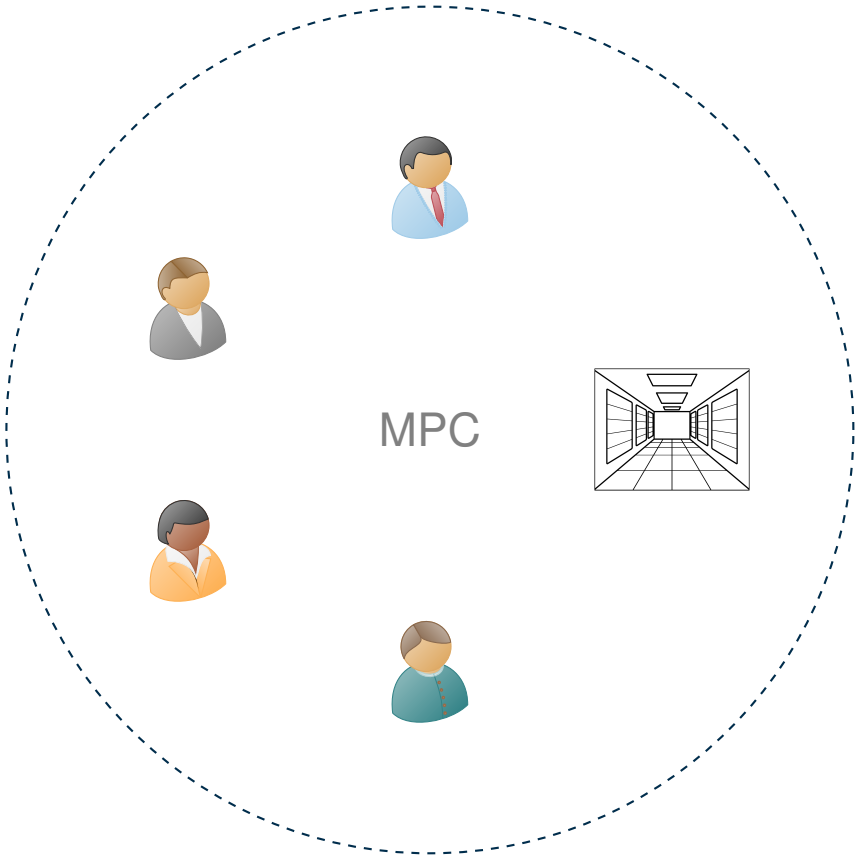
Universal Composability

With Global Subroutines



$\mathcal{G}$ is a global functionality: $\mathcal{Z}$ sees the same instance in both worlds.

Single-Enclave MPC



Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]

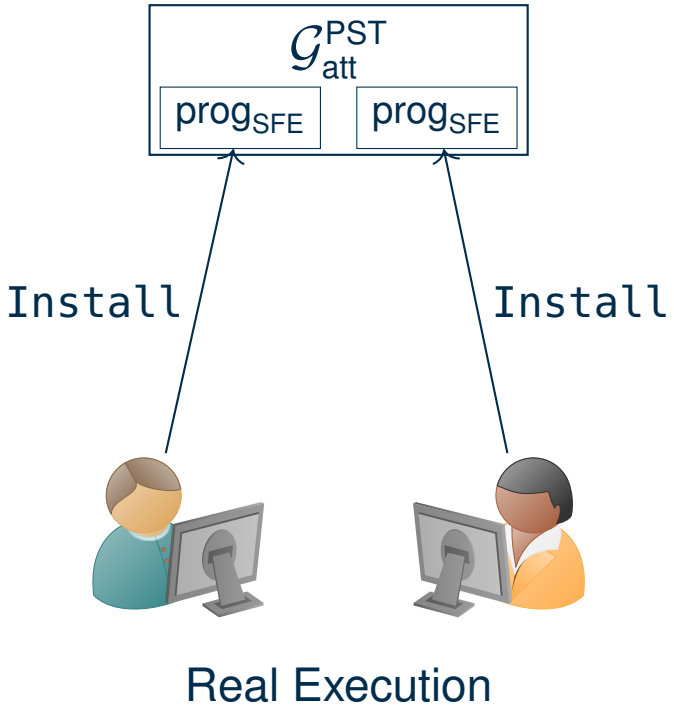




Real Execution

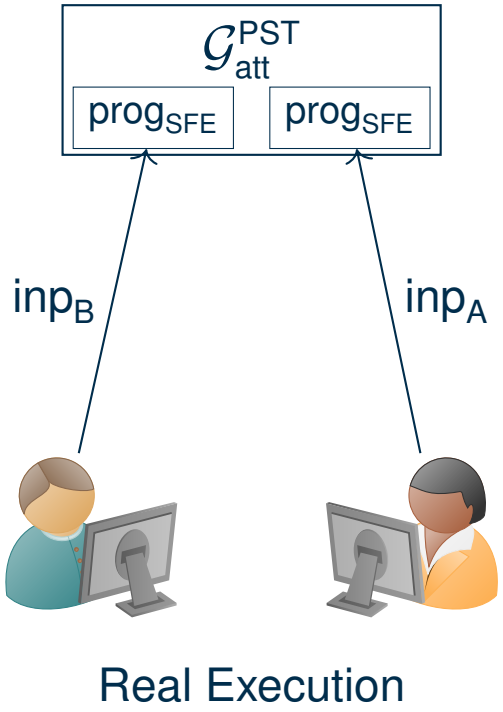
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



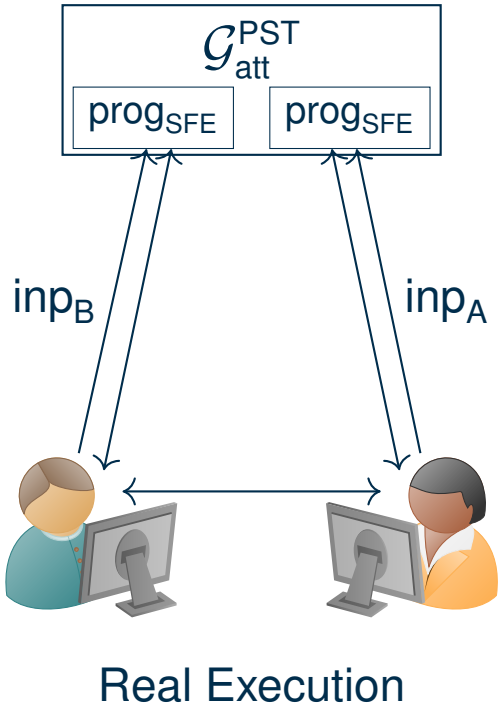
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



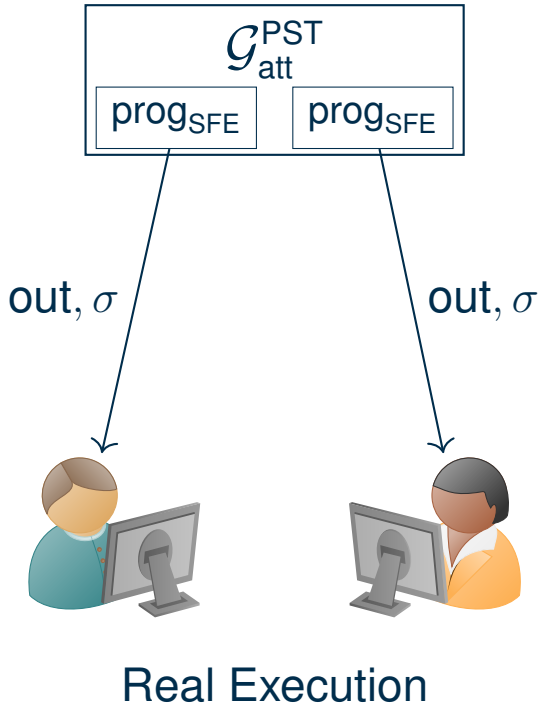
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



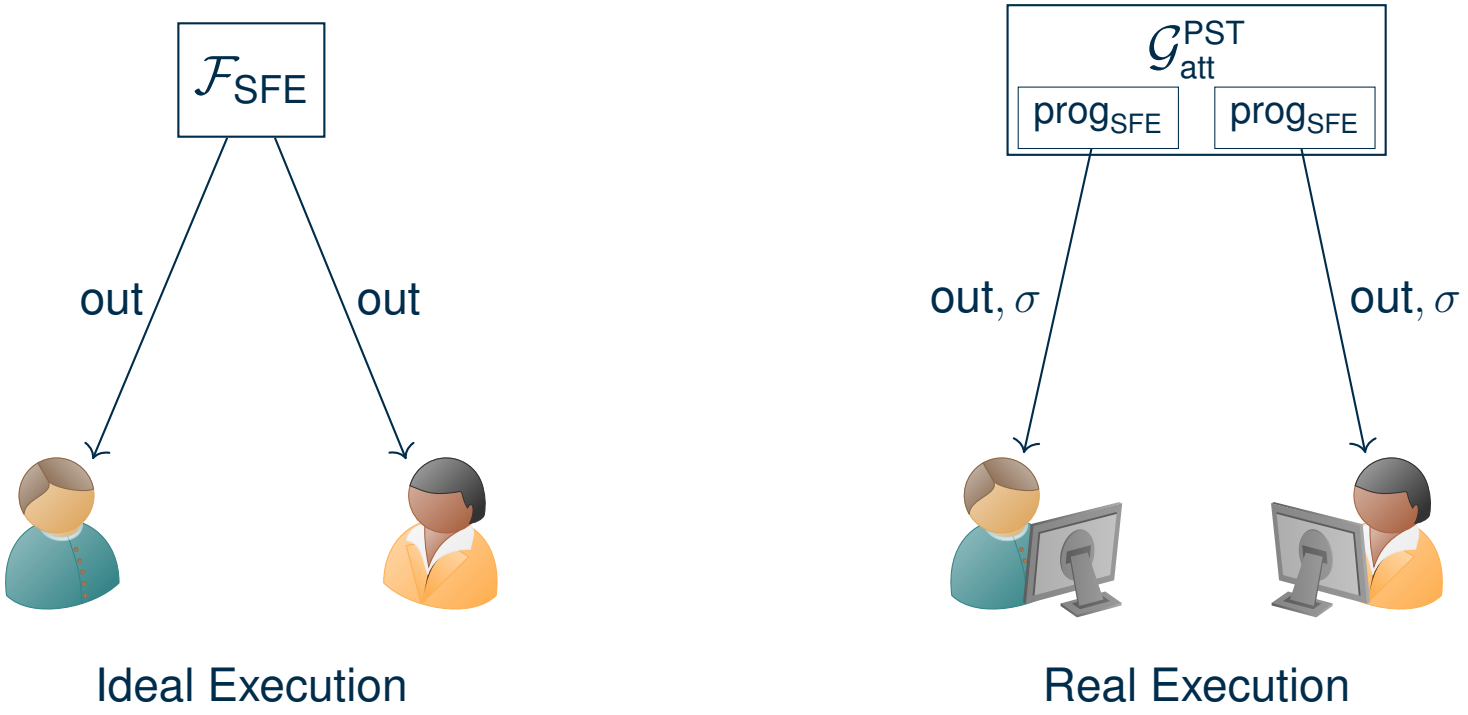
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



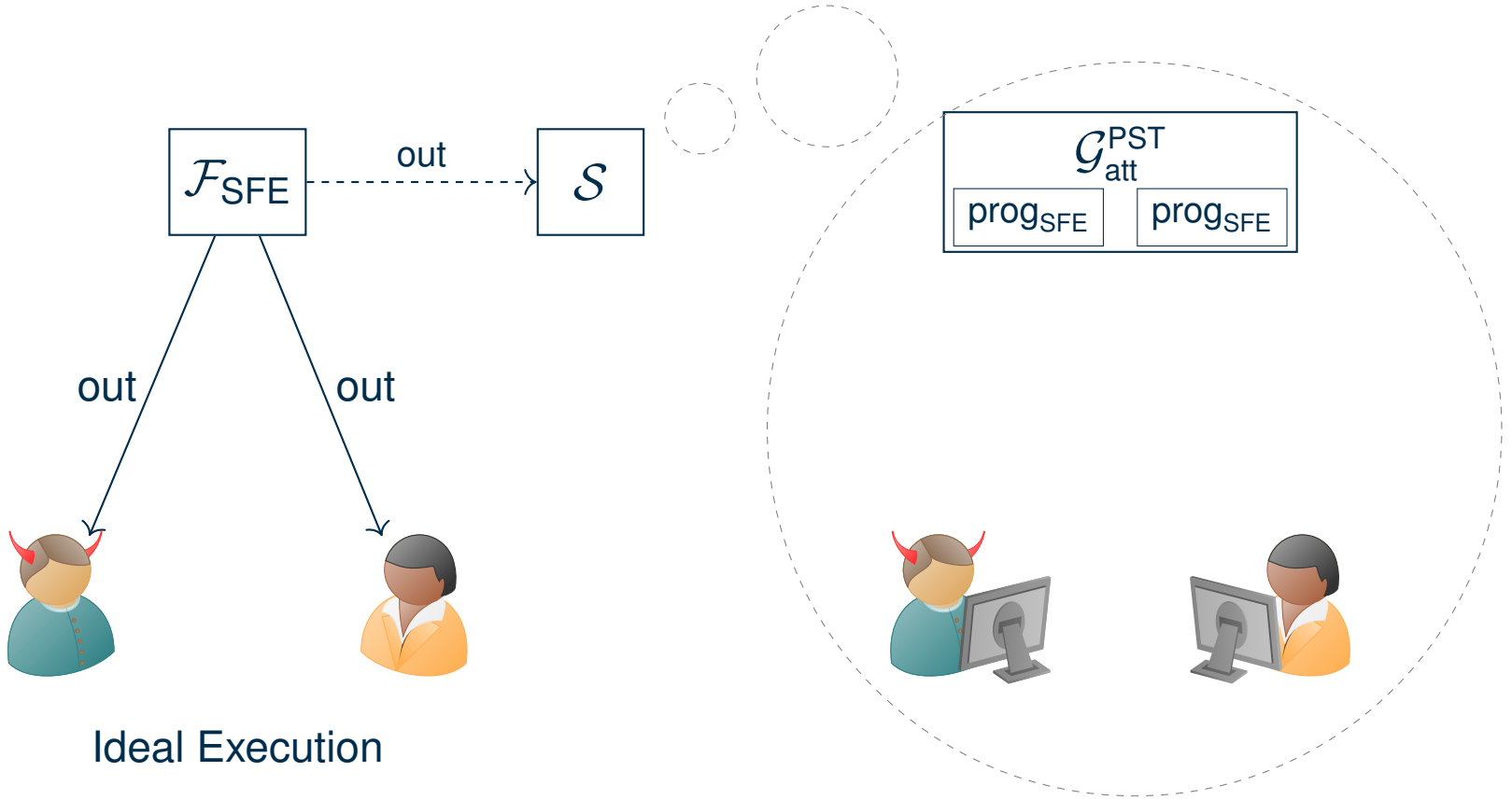
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



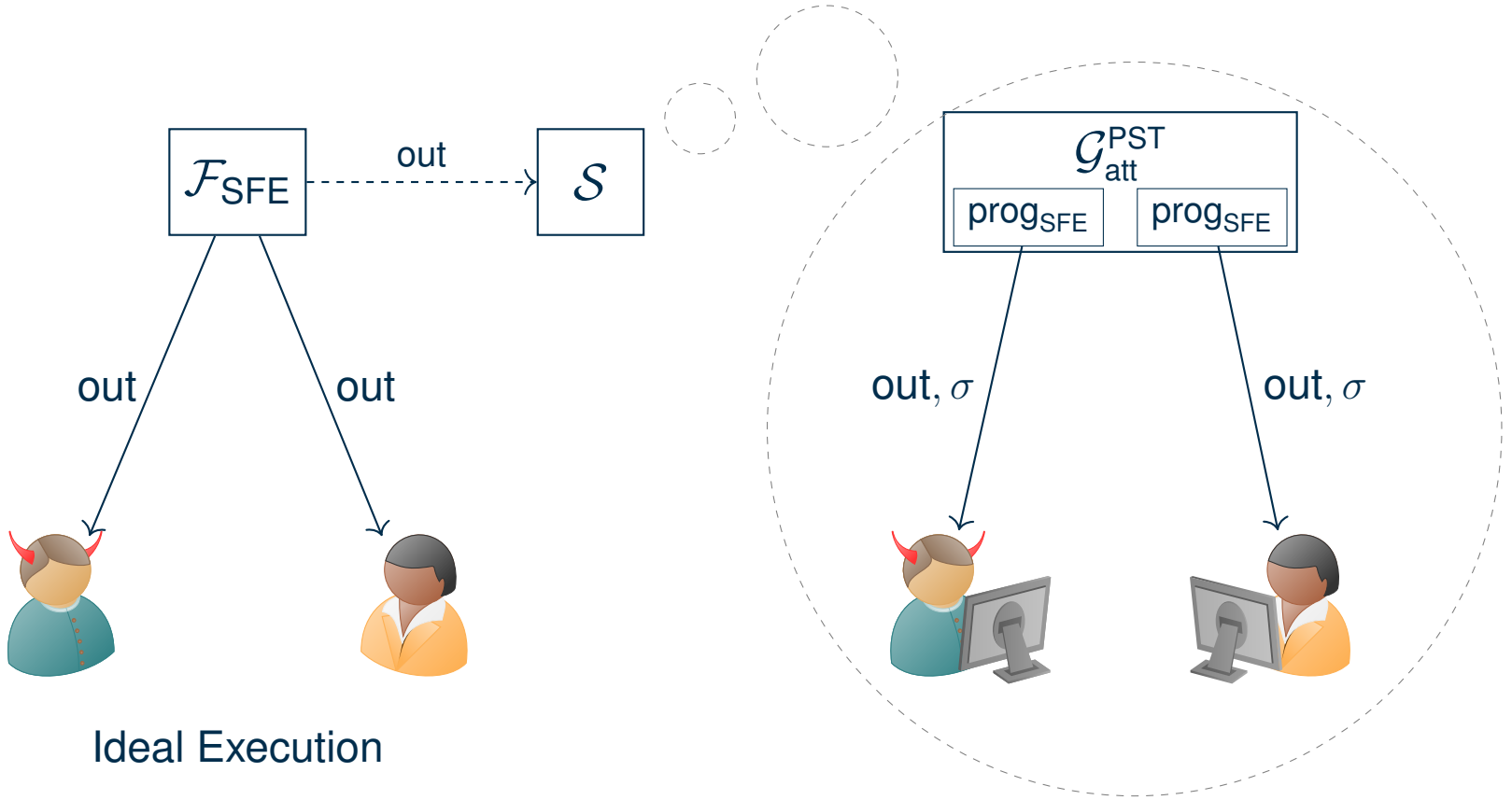
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



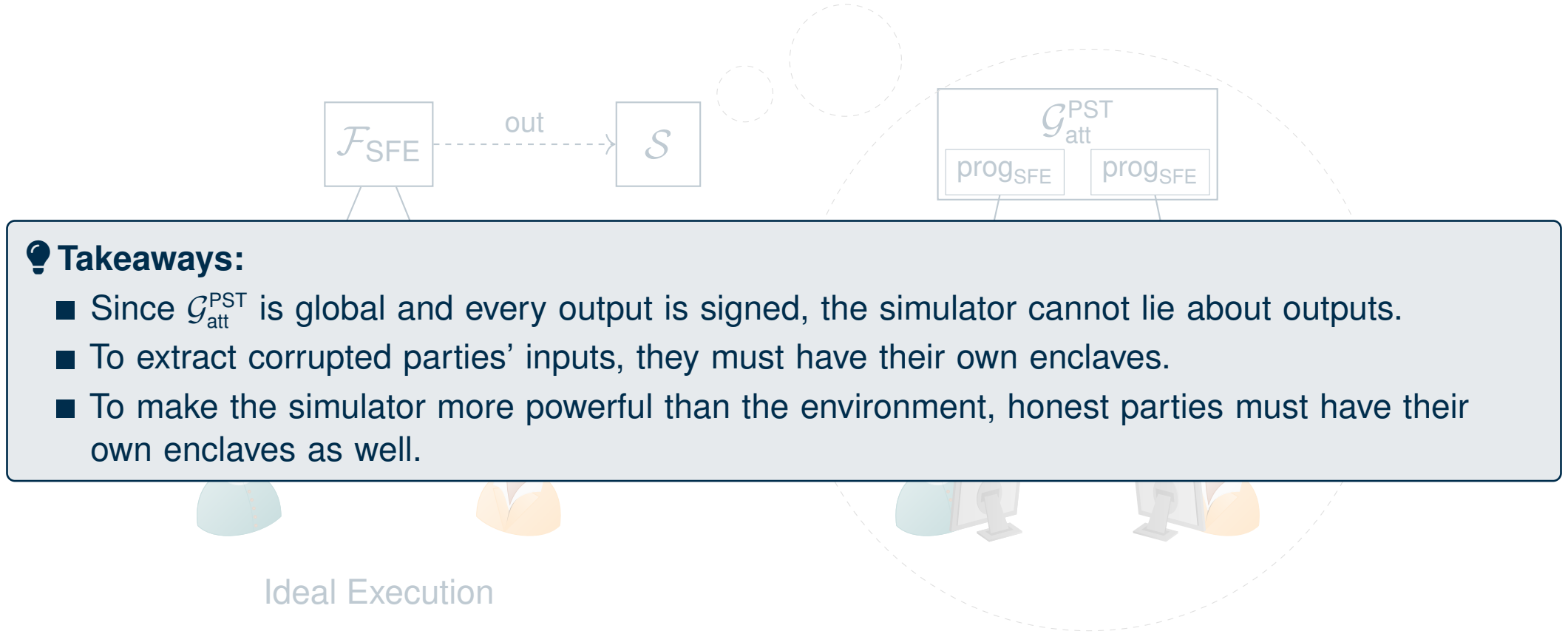
[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Impossibility of n-Party MPC with $n-1$ Enclaves in Current Enclave Models[2]



Takeaways:

- Since $\mathcal{G}_{\text{att}}^{\text{PST}}$ is global and every output is signed, the simulator cannot lie about outputs.
- To extract corrupted parties' inputs, they must have their own enclaves.
- To make the simulator more powerful than the environment, honest parties must have their own enclaves as well.

[2]Pass, Shi, and Tramèr 2017; Martinico and Kohlweiss 2025; Dörre, Mechler, and Müller-Quade 2023.

Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct






Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct



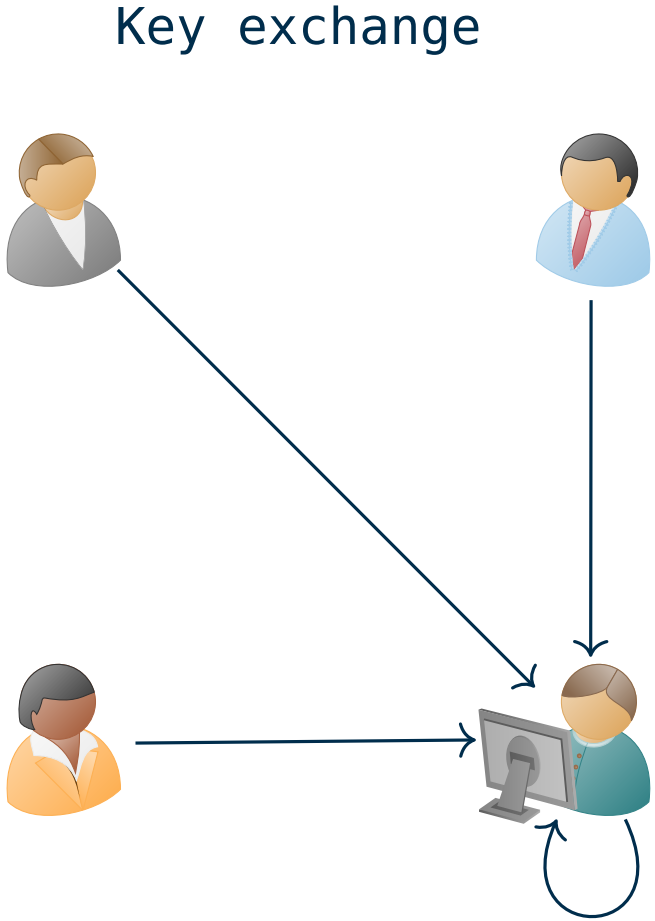



Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct

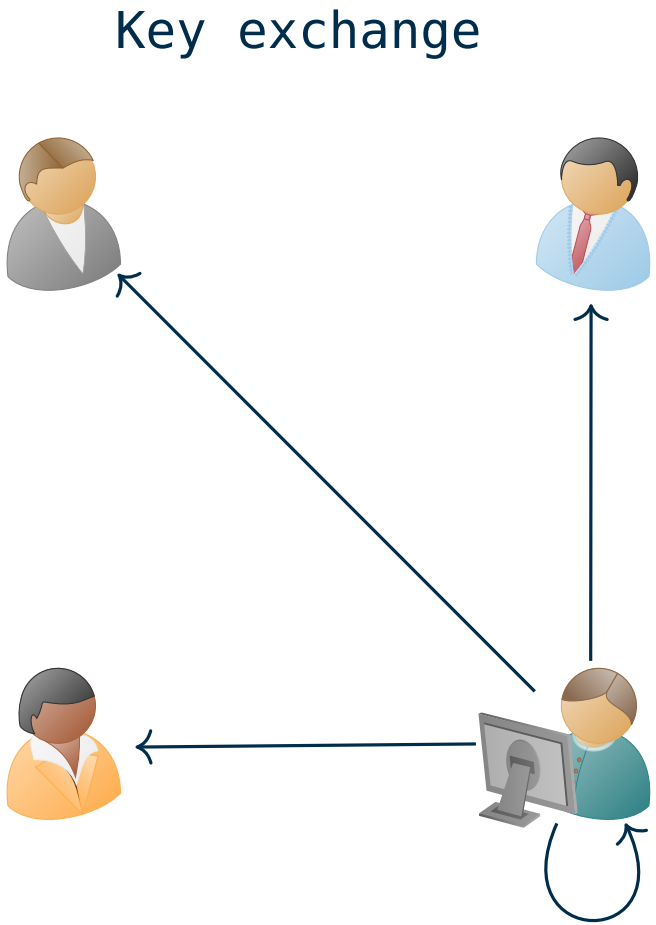


Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct



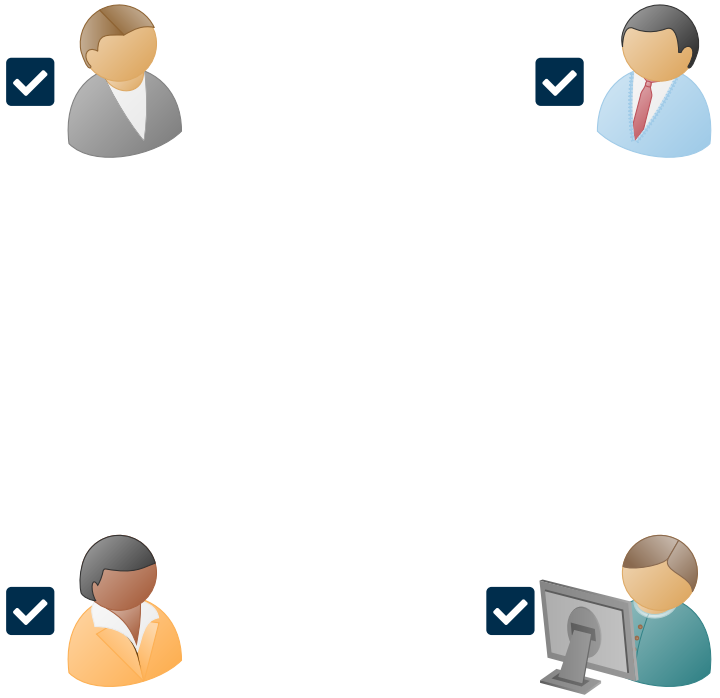
Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct

Key exchange

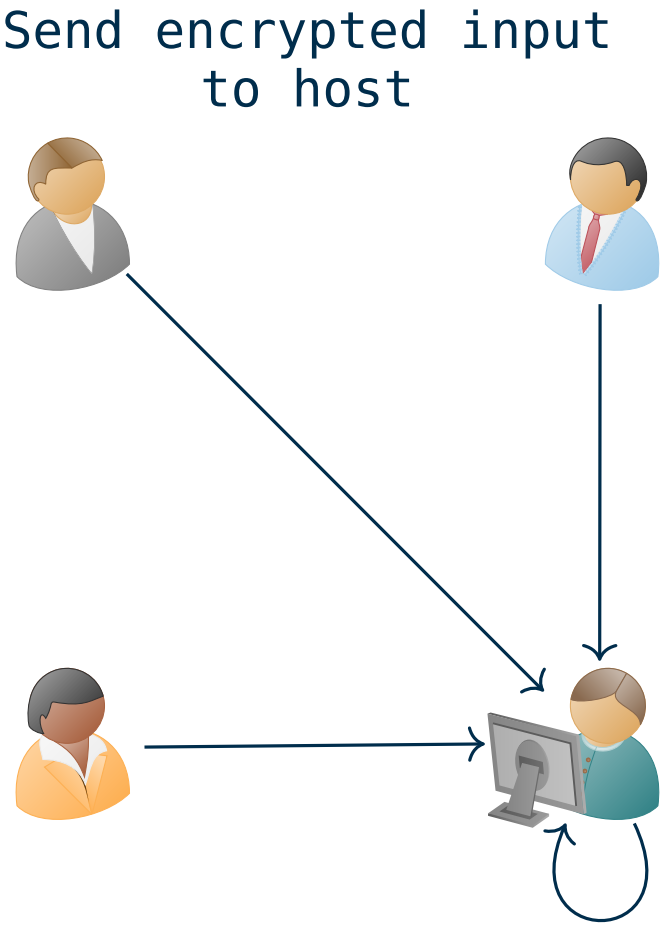


Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct



Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct



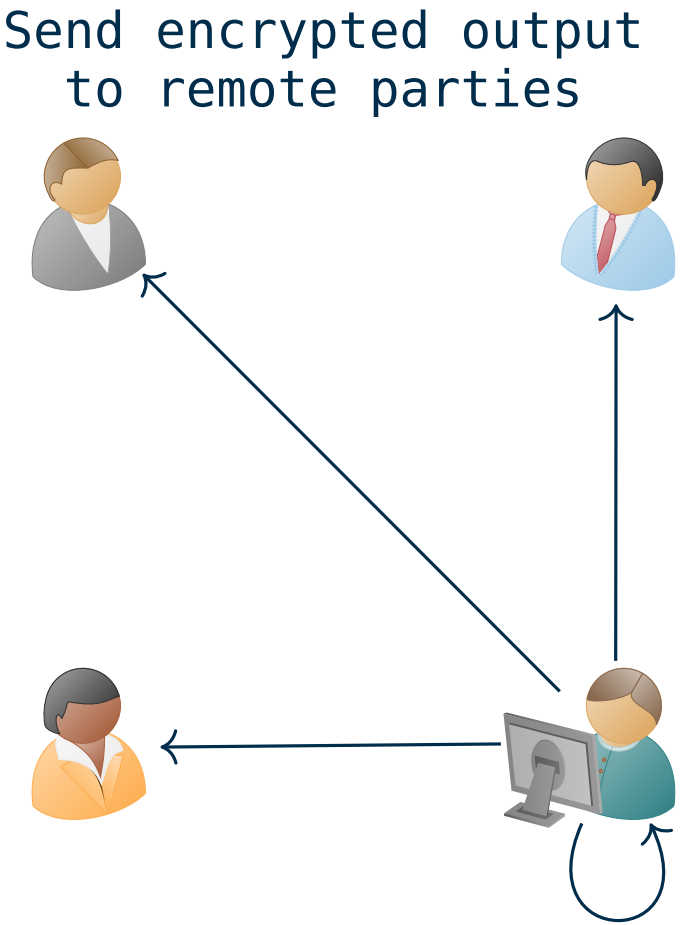



Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct



Our Solution for Single-Enclave MPC

$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`VerifyAttestation`(aid, out) from P: return 1 if aid belongs to P
`VerifyAttestationAll`(aid, out): return 1 if aid correct

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct






Our Solution for Single-Enclave MPC

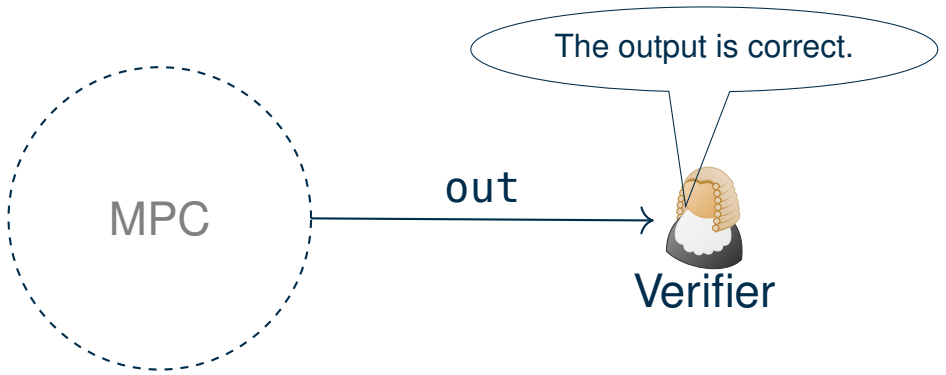
$\mathcal{G}_{\text{att}}$

`Initialize` enclave
`Install`(prog): return *eid*
`Resume`(inp): execute prog on inp and return out
`Verif`
`Verif`

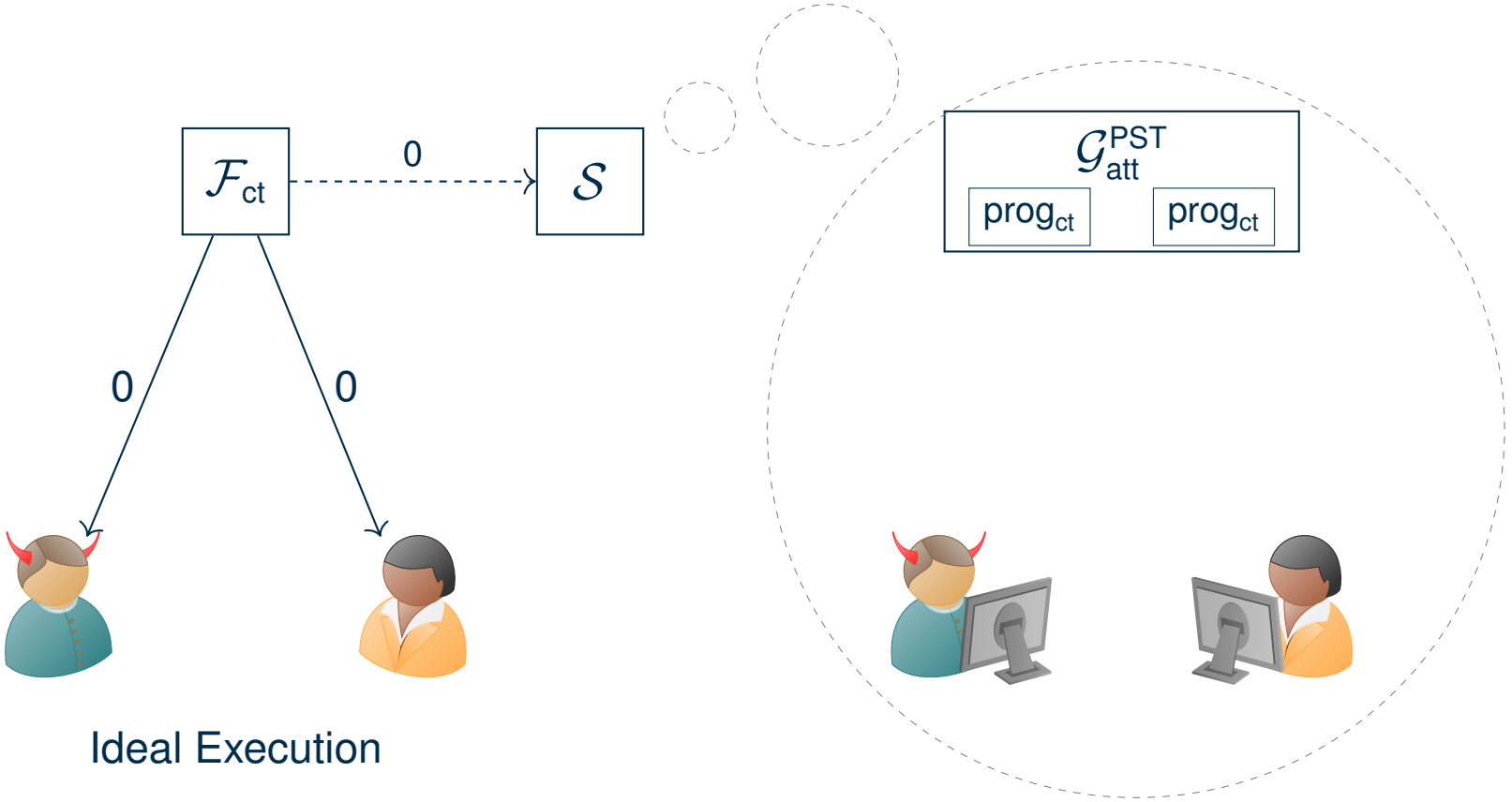
Takeaway:
Designated-verifier attestations allow for per-party trapdoors for extraction and equivocation.

Enclave-internal interfaces:
`AddAttestation`(P, out) from enclave: return aid for P if out correct
`AddAttestationAll`(out): return aid if out correct

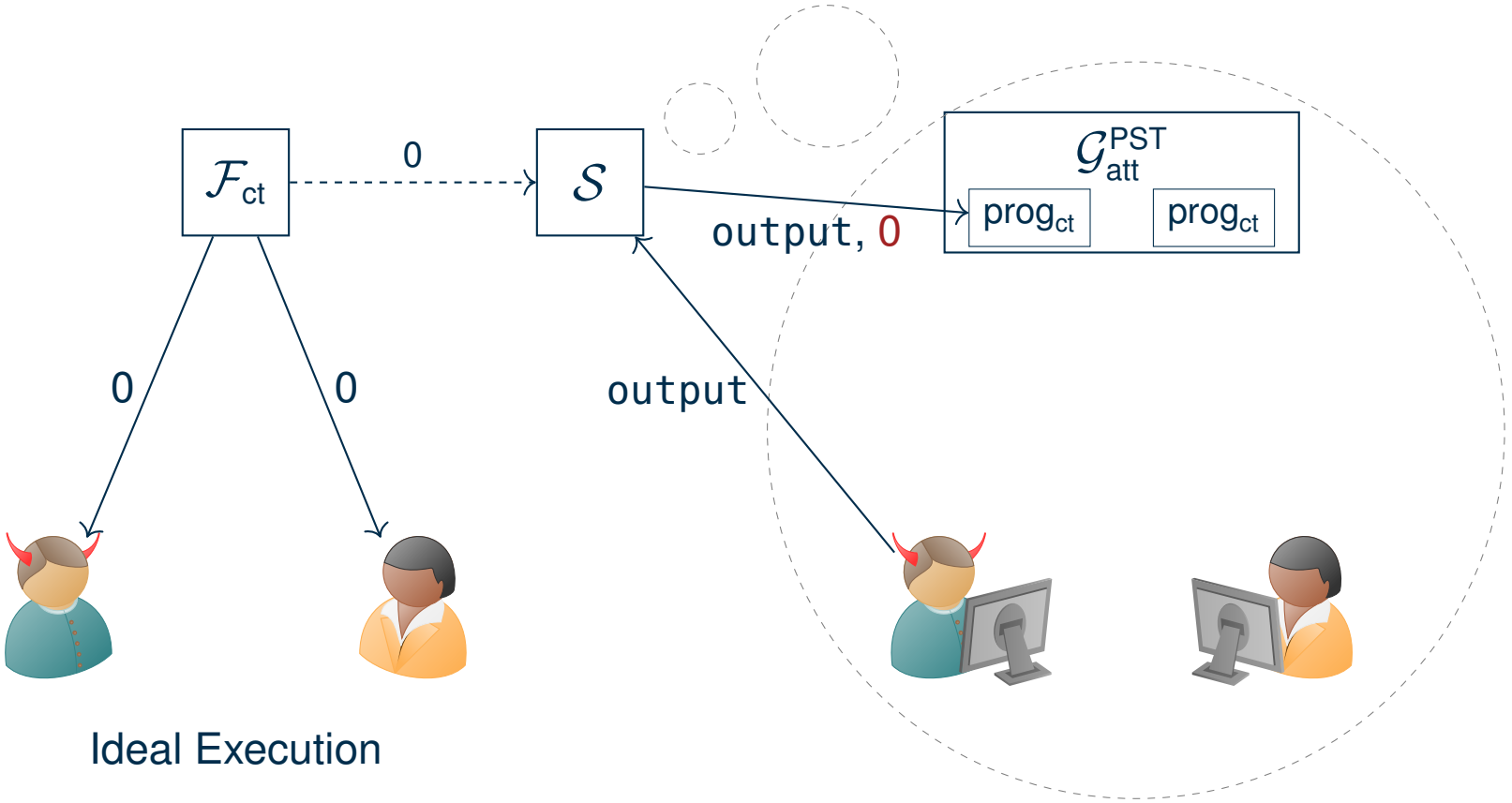
Public Verifiability



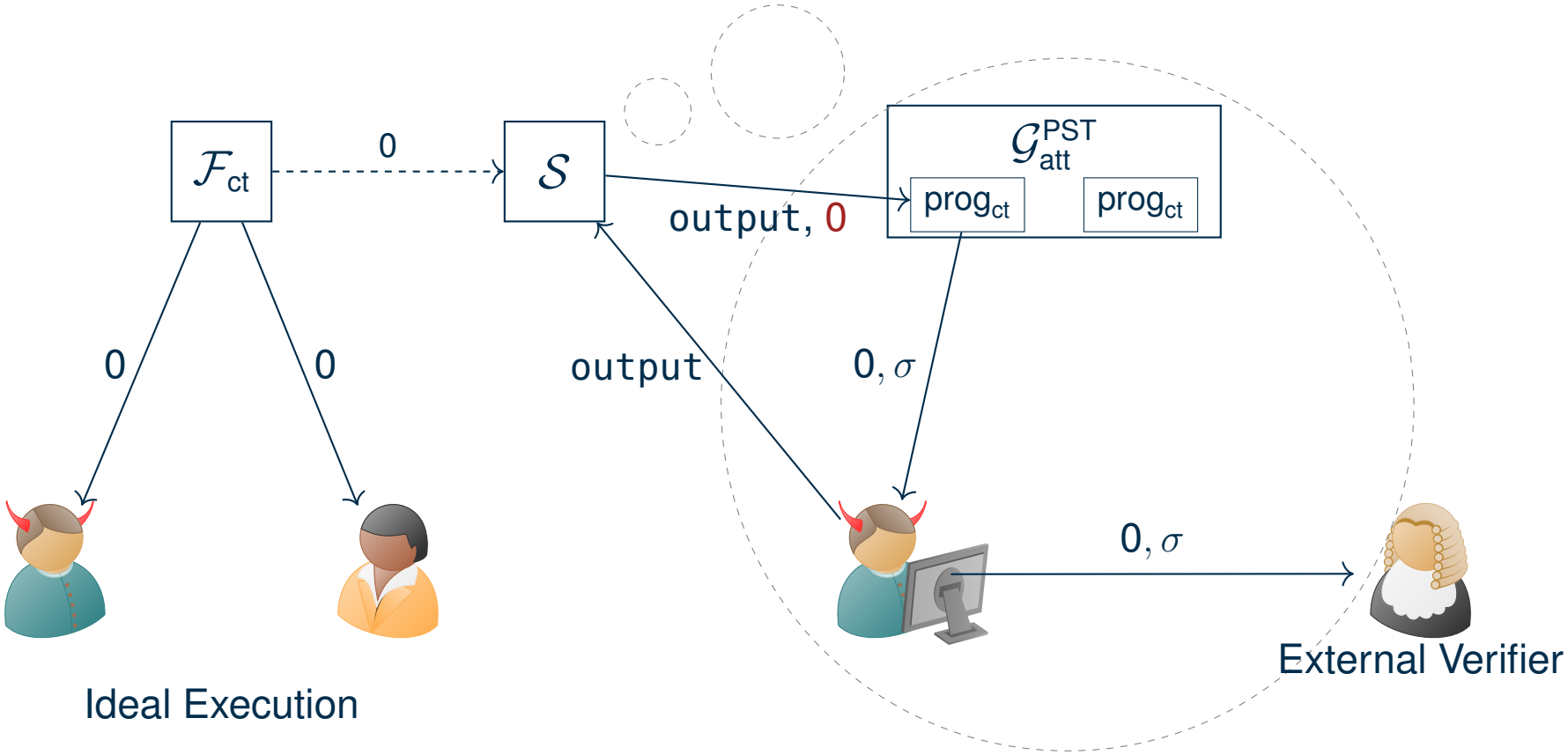
Impossibility of Public Verifiability in Current Enclave Models



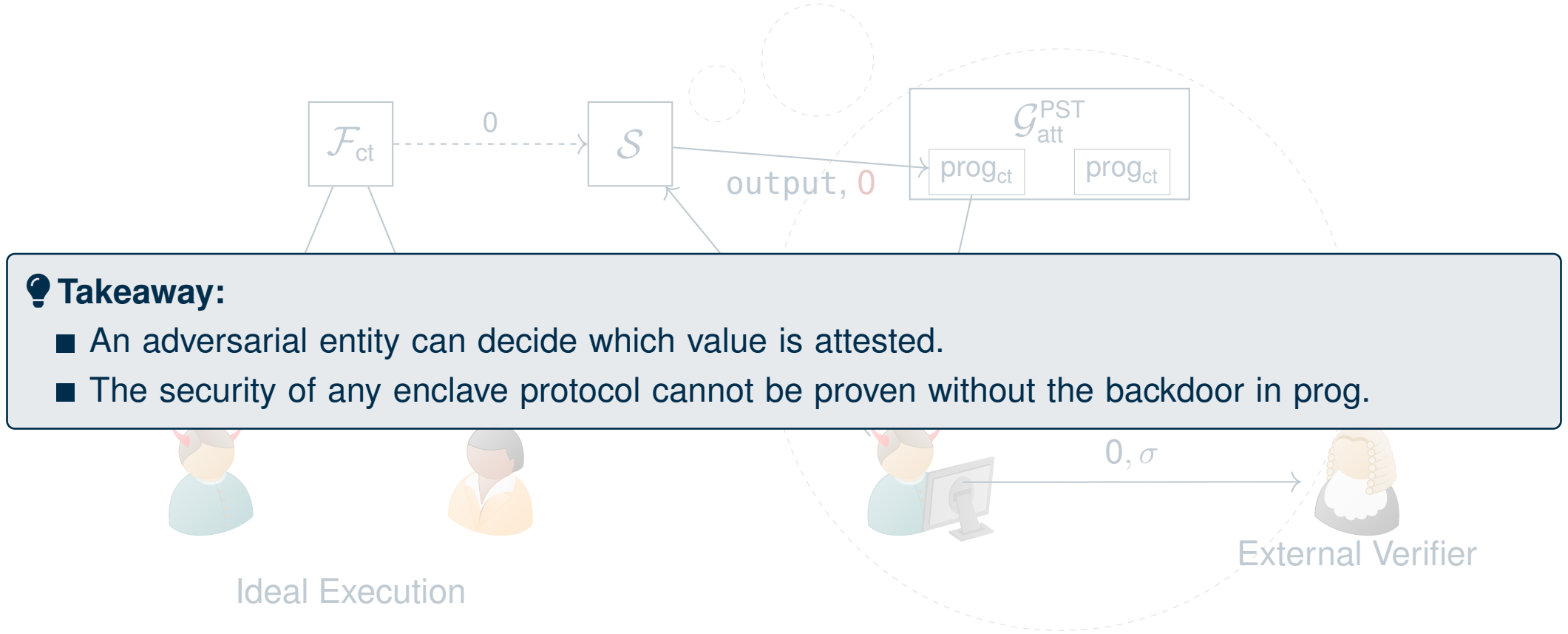
Impossibility of Public Verifiability in Current Enclave Models



Impossibility of Public Verifiability in Current Enclave Models



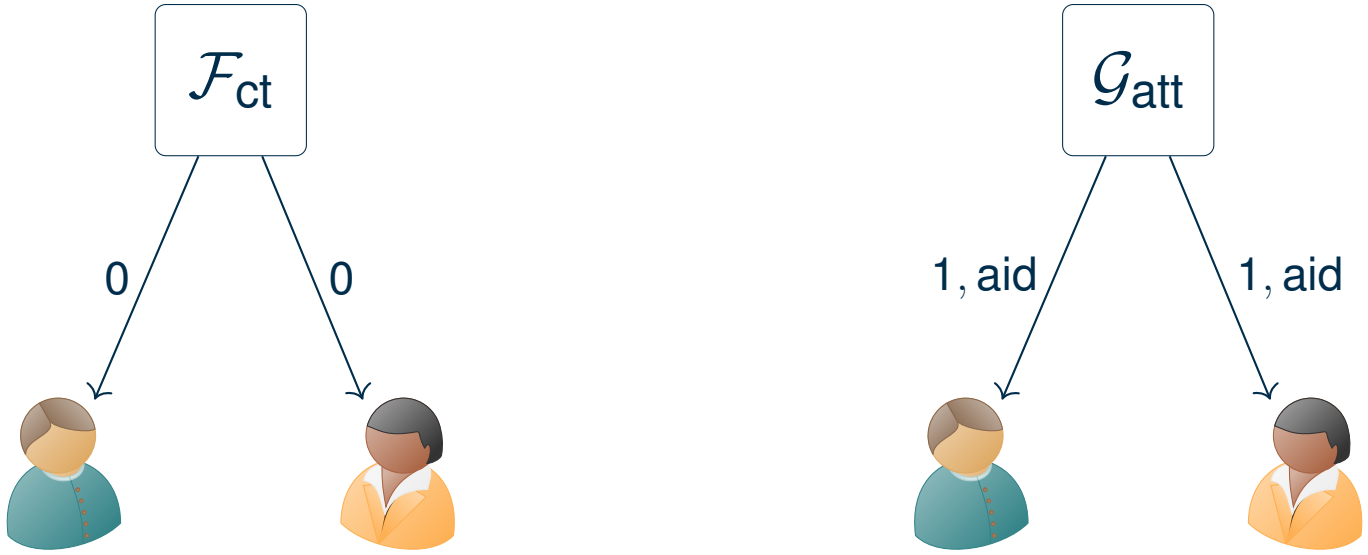
Impossibility of Public Verifiability in Current Enclave Models



Takeaway:

- An adversarial entity can decide which value is attested.
- The security of any enclave protocol cannot be proven without the backdoor in prog.

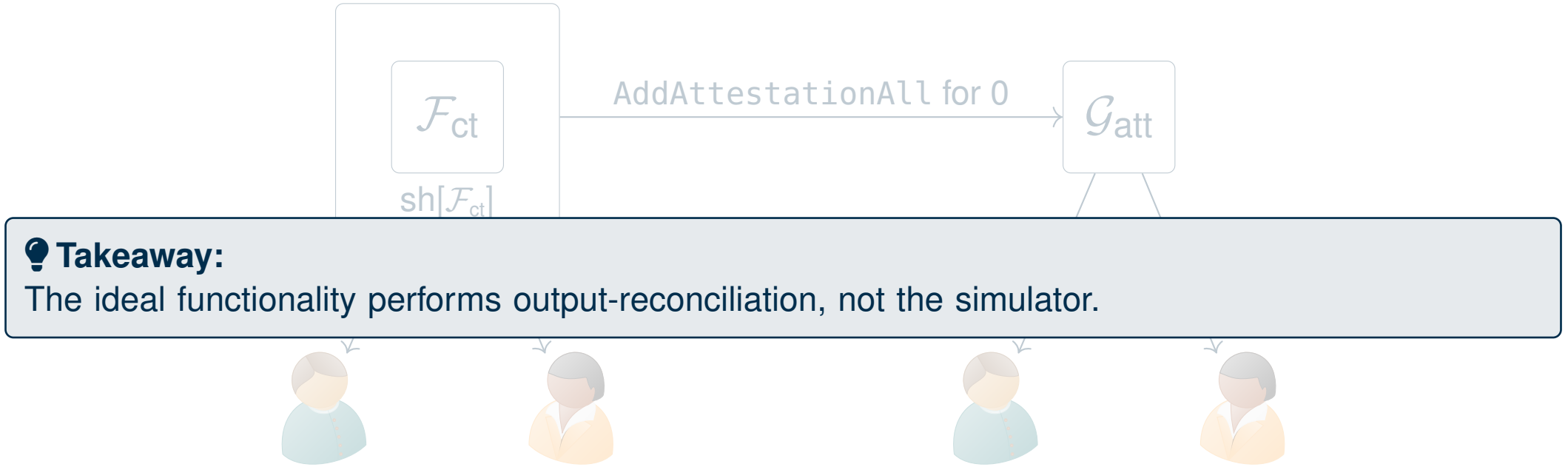
Our Solution to Achieve Public Verifiability



Our Solution to Achieve Public Verifiability



Our Solution to Achieve Public Verifiability



Takeaway:
The ideal functionality performs output-reconciliation, not the simulator.

Conclusion

- What are our results?
  - Single-enclave MPC via designated-verifier attestations
  - Public verifiability in UC via output-reconciliation of the ideal functionality

Conclusion

- What are our results?
  - Single-enclave MPC via designated-verifier attestations
  - Public verifiability in UC via output-reconciliation of the ideal functionality
- What else is there?
  - Instantiation of $\mathcal{G}_{\text{att}}$ with MACs and signatures instead of information-theoretically secure verification at $\mathcal{G}_{\text{att}}$
  - Proof that $\mathcal{G}_{\text{att}}^{\text{MAC}}$ emulates $\mathcal{G}_{\text{att}}$

Thank you!

- What are our results?
  - Single-enclave MPC via designated-verifier attestations
  - Public verifiability in UC via output-reconciliation of the ideal functionality
- What else is there?
  - Instantiation of $\mathcal{G}_{\text{att}}$ with MACs and signatures instead of information-theoretically secure verification at $\mathcal{G}_{\text{att}}$
  - Proof that $\mathcal{G}_{\text{att}}^{\text{MAC}}$ emulates $\mathcal{G}_{\text{att}}$

Extended version available at:
`https://ia.cr/2026/1005`

KASTEL - Institute of Information Security and Dependability